Assignment 2

By Attila Garganego (22339628)

1. Abstract

FSMs are used in many applications of today from electronic passcode locks to compilers. FSMs are important for controlling the behaviour of systems, according to the state it's in and the conditions it needs to act upon. In this lab I combined FSMs with LFSRs and a counter which I have built in the previous lab F to create a codeword counter. This was then displayed using lab G seven segment display code. This was all coded through Vivado and implemented using a Basys 3 FPGA board.

2. Introduction

The purpose for this lab was to build a codeword counter by using old modules from previous labs. This counter's logic had to be rising edge sensitive and to reset to a button or lever input. It was then required to display the counter's output through the Basys 3 seven segment display. This then must be tested through a module testbench before being implemented on the board.

Finite State Machine (FSM) - is a coding/electronic logic that dictates the transition associated between states according to its inputs at the rising edge and determines how many states it can have.

Linear Feedback Shift Register (LFSR) – is a shift register that determines its new least significant bit (in this case), by using functions whose input bits come from two or more bits from the previous state also called taps.

3. Method

3.1. Creation of Code

To create the code word counter the first thing I did was to create the FSM module, this was done by editing the counter code that I used in lab F. Most of the code was changed in that module to create the FSM. This was then tested with a testbench before going onto the next module.

I then imported the lab G stopwatch code this also had some changes. The clock divider used in the module was removed and an if statement was inserted that checks if the word counter value is bigger than the displayed value.

Then the LFSR from lab F had the: taps, tap length and seed value changed for the new values requested in the lab.

The changes to the LFSR and counter were then tested with another testbench when the outputs from the testbench matched the expected outputs I moved on.

A top module was then made to connect the LFSR and counter with the FSM.

The clock divider from lab F was then imported into the top module with the seven-segment display module from lab G. Both weren't changed except the dp input for the seven-segment display was set to all 1's.

Then the top module was tested, and the simulation outputs were compared with the expected inputs when the output matched.

The top module was synthesized, and constraints were made. After that the design was implemented on the board and tested.

3.2. Testbenches

3.2.1. Top module testbench

My top module testbench, has very little tests as the only inputs are reset and reg_check. So first I tested resetting my top module making sure everything reset properly. Then I tested changing the registry as it only shows the first 11 bits i.e. LEDs outputs. I then let it run checking if it could count the code word properly, if it could reset and if it registered looping of the LFSR.

Time (0.2µs = clk)	Reset (input)	Reg_check (input)	LFSR **(output)	max_val (output)	An * (output)	seven_seg * (output)	Counter (internal)
0-1	1	0	00001101000	0	1110	10000001	0
1-2	0	0	00011010001	0	~	10000001	0
2-3	1	0	00001101000	0	1110	10000001	0
3-229	0	1	~	0	~	10000001	0
230	0	1	-0000001000	0	1110	10000001	1
231	0	1	-0000010000	0	1110	11001111	1
468	0	1	-0110111000	0	1110	10010010	3
469	0	1	-1101110000	0	1110	10000110	3
470	0	1	-1011100001	0	1110	10000110	4
471	0	1	-0111000010	0	1110	11001100	4

*displaying only the important bit vector of seven seg as it cycles between the 4 different displays.

** The LFSR shows the current reg not the past reg.

The testbench shows that there's a 1 clock cycle before the display updates to the new counted value.

3.2.2. Counter Testbench

The other test bench modules are the ones I made or significantly edited. So, the clock divider and seven segment display modules weren't tested.

The testbench design for the counter module was to test if it could follow the FSM states properly and could count properly. The reset was first tested, then the enable was tested so if the lfsr stopped it wouldn't start feeding in more values. Then it was tested if it could detect the codeword and different edge cases that should turn the count as high.

re_set	enable	Msb (bits fed into fsm)	Count_1
1	0	0	0
0	0	010101	0
0	1	010101	1
0	1	01010101	2
0	1	1010101	1
0	1	0010101	1

The fsm should follow the states like in the diagram, with the red lines showing where the state should transition if the condition isn't met and if the state is met the black lines show the continuation.

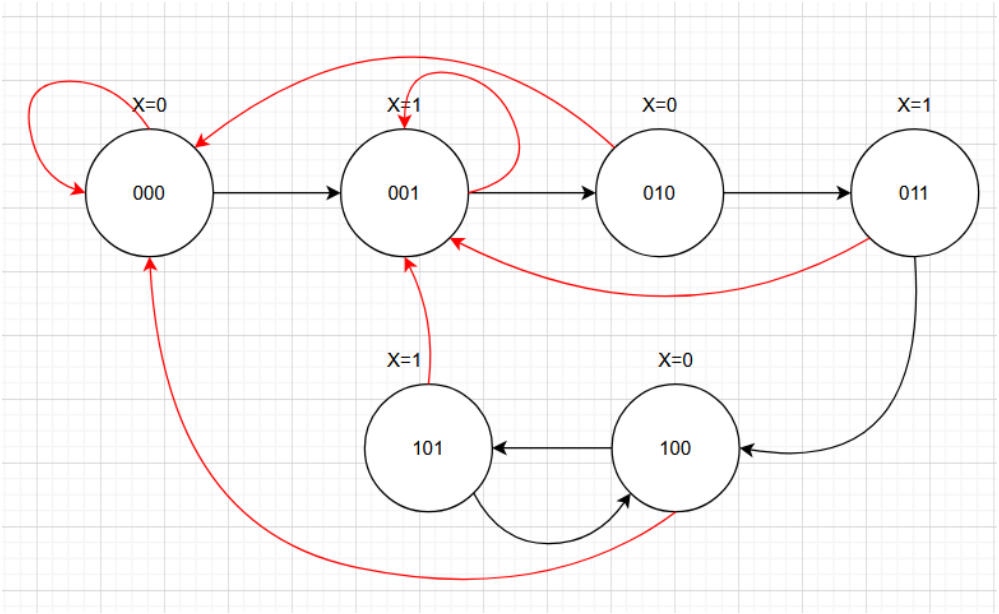


Figure 1: FSM states

3.2.3. Hex to binary decoder

The testbench design for the stop_watch_if was if it could decode the counter input to decimal values. First thing to test was to see if it would reset properly. After that it was to test if the d_out values would increase or change without the counter increasing then it was tested the counter increasing slowly or quickly to see if it would increase slowly like it's expected to do.

Clock cycle	re_set	counter	d3_out	d2_out	d1_out	d0_out
0-10	1	0	0	0	0	0
11-12	0	0	0	0	0	0
13	0	1	0	0	0	0
14	0	1	0	0	0	1
15	0	4	0	0	0	1
16	0	4	0	0	0	2
17	0	4	0	0	0	3
18	0	4	0	0	0	4
1019	0	1000	1	0	0	0

3.2.4. LFSR testbench

For the LFSR testbench I wanted to test the counter and the LFSR capabilities to stop with the enable pin and to test the reset. So reset and enable were the first to be tested, then the LFSR was let run 21^n-1 clock cycles so the mx_out led could be seen turn on and also to test how many times the word can be counted in the LFSR output. It was then reset to make sure everything reset properly, and it was run again.

Clock cycles	re_set	enable	Q_out *	count	mx_out
1	1	0	104	0	0

2-6	0	0	104	0	0
7	0	1	209	0	0
7- 2097158	0	1	~	~	~
2097159	0	1	104	32768	1
2097160 - 2097164	0	0	104	32768	1
2097165	1	1	104	0	0
4194316	0	1	104	32768	1

*20-bit vector too long so unsigned decimal will be used to represent the 20-bit vector

3.2.5. Clock control

The clock is controlled using the clock divider module. This module counts to 50 million it then flips the output bit. This bit output is the clock used in the rest of the module except for the seven-segment display. The clock divider is then connected to the 100MHz crystal oscillator clock (w5), this is to slow down the clock to a manageable 1 Hz.

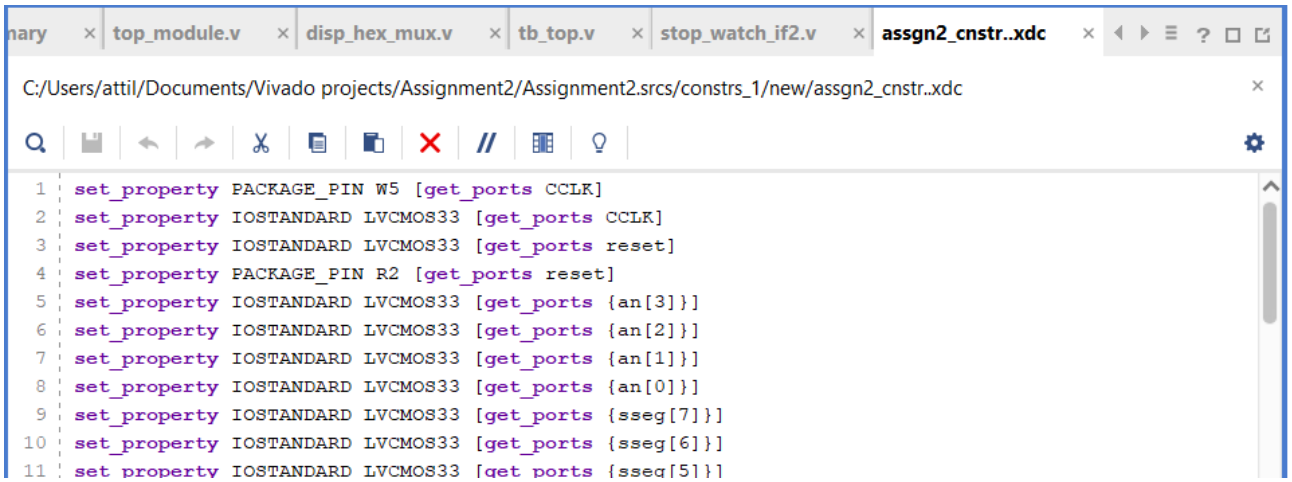
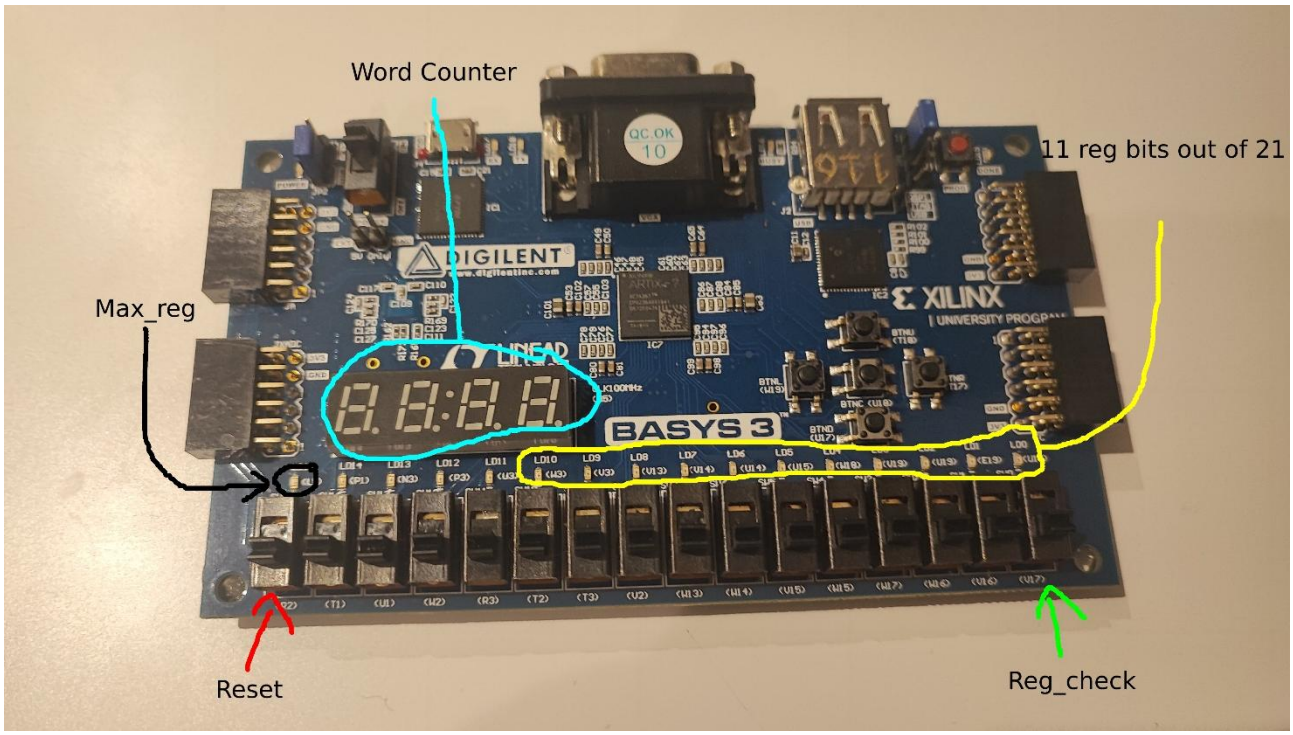


Figure 2: Constrains file where COC is connected

3.3. Demo

The Basys-3 board implemented design works like this:

- There are two switch inputs:
  - The reset switch is R2
  - The reg_check switch is V17. What is does is display the 11 LSBs of the LFSR reg when low and when high it displays the 10 MSBs of the LFSR reg.
- There are two LED outputs:
  - The LD10-LD0 represent the LFSR's current reg. When the reg_check is low all LEDs can be looked at, but when reg_check is high LD10 should be ignored as we only display 10 bits instead of 11.
  - LD15 represents the max_reg and will turn on only when the LFSR loops.
- The seven-segment display instead shows the number of words counted from the basys-3 board



4. Results

4.1. Waveforms

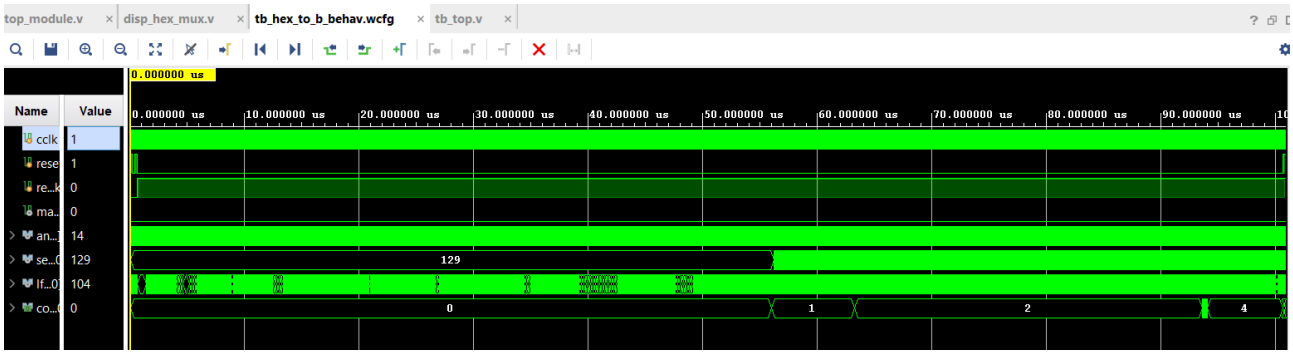


Figure 3: Top module testbench

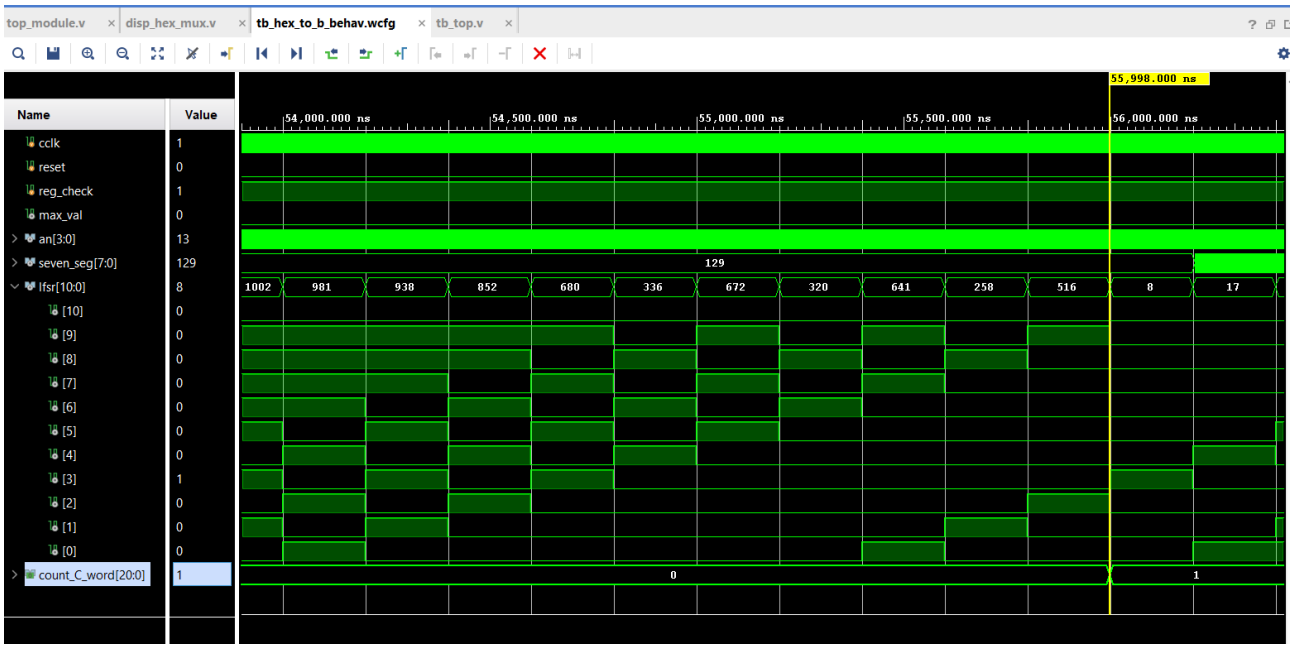


Figure 4: Top module testbench counter going to 1. (counter is an internal signal)

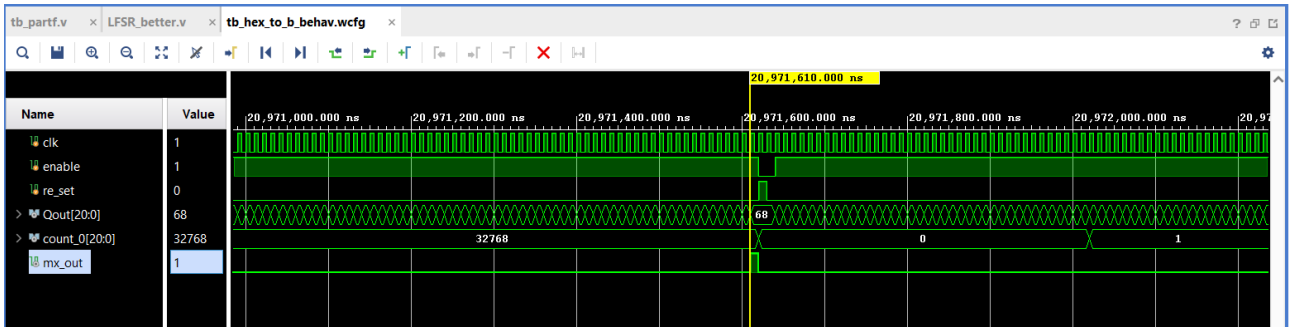
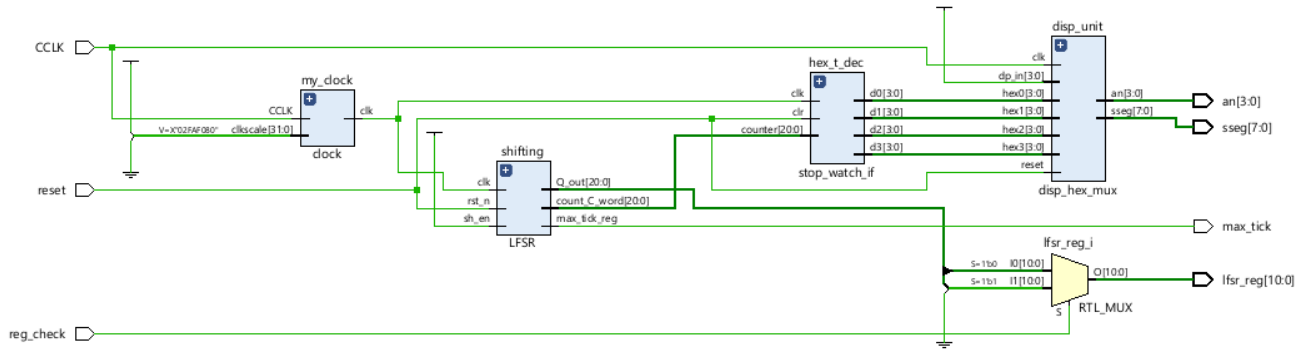
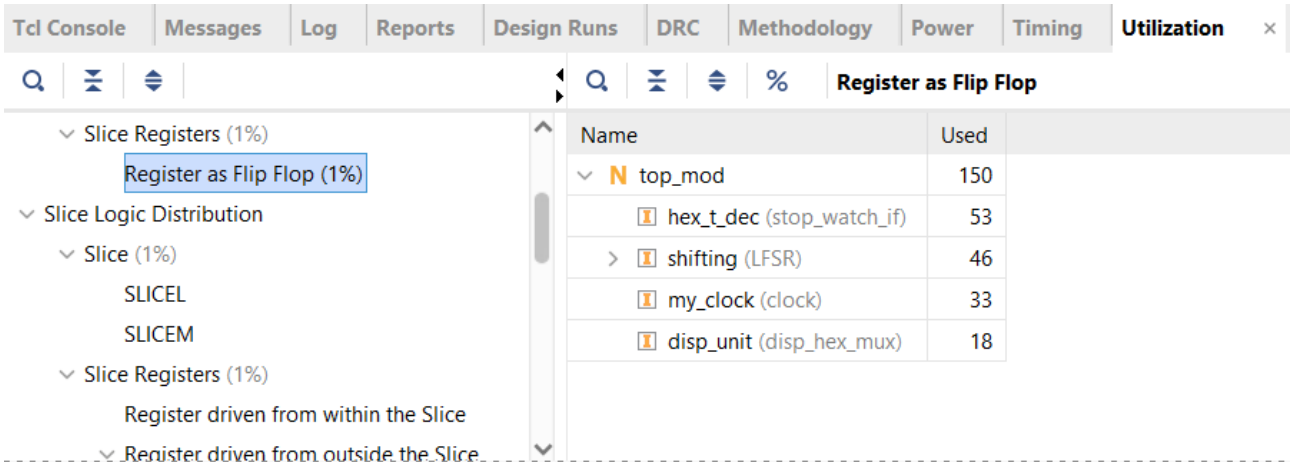


Figure 5: To find max combinations of the codeword LFSR testbench was used

4.2. Schematic of modules



4.3. Utilization flipflop



4.4. Hierarchy sources

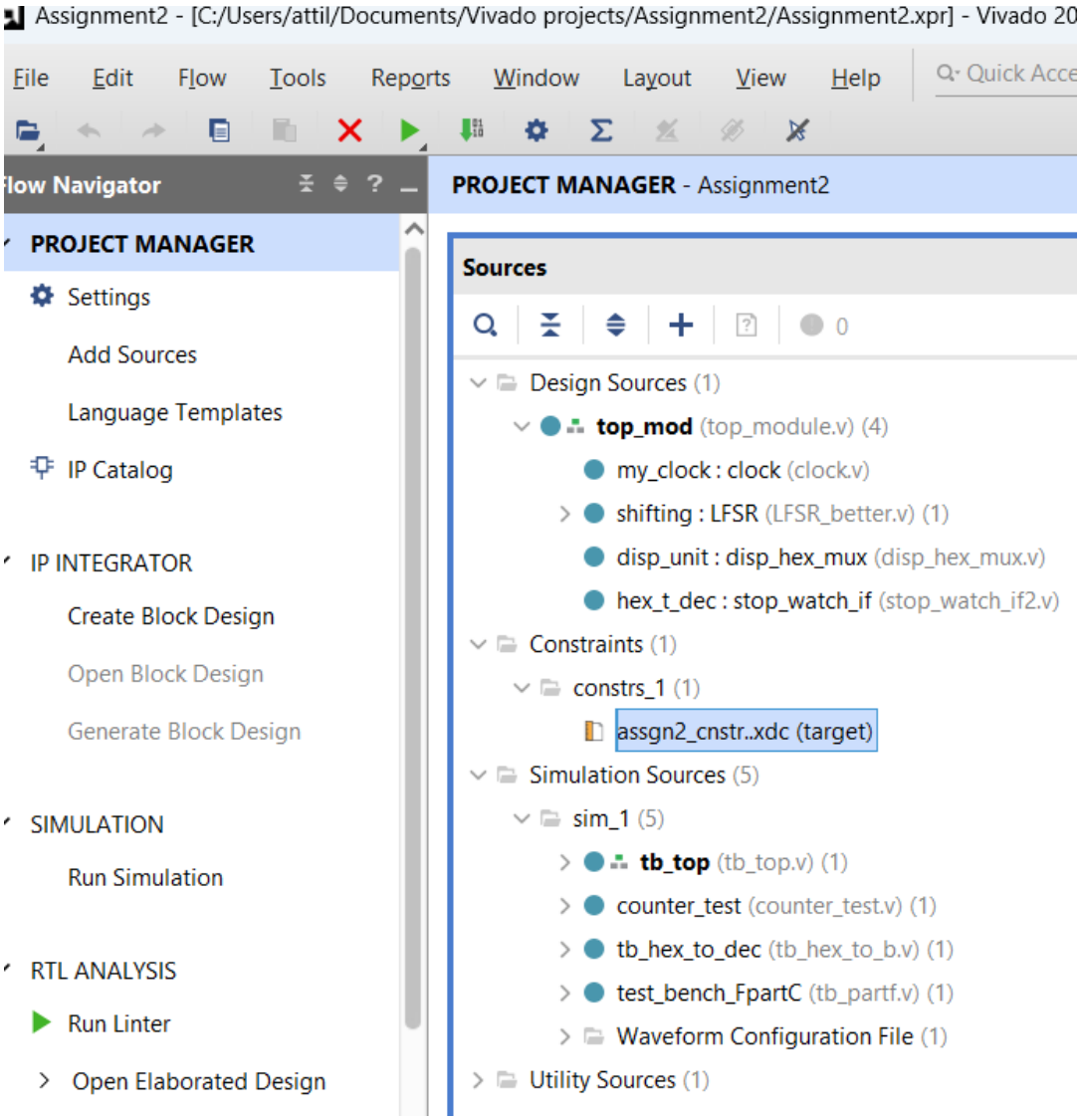


Figure 6: Sources hierarchy

5. Discussion

In this lab the FSM was implemented successfully with the LFSR. The display does have a delay of 1-2 clock cycles I believe this is probably caused by the stop_watch_if module as it converts my bits from hex form to decimal. First the next registry bits need to update once that happens a clock later the actual registry updates this causes a delay of 1 clock cycle. The rest of the implementation works perfectly fine. The system is synchronous and reset works on the basys-3 board when tested.